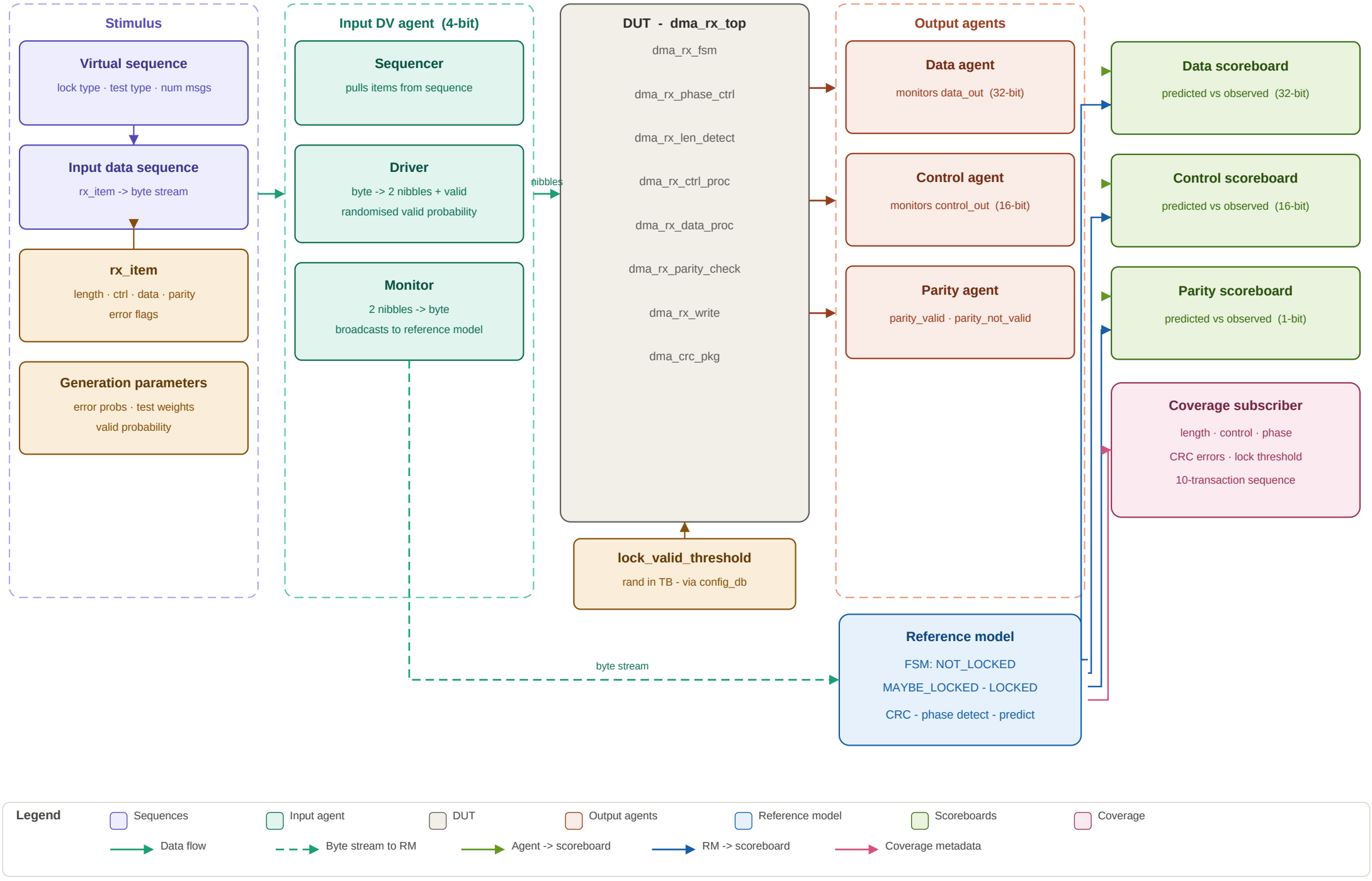
# RX DMA — Verification Plan

UVM Verification Environment | Version 1.0

Project: rx_uvm_environment | Enics | April 2026


Stimulus
Virtual sequence
lock type · test type · num msgs
Input data sequence
rx_item -> byte stream
rx_item
length · ctrl · data · parity
error flags
Generation parameters
error probs · test weights
valid probability
Input DV agent (4-bit)
Sequencer
pulls items from sequence
Driver
byte -> 2 nibbles + valid
randomised valid probability
Monitor
2 nibbles -> byte
broadcasts to reference model
nibbles
DUT - dma_rx_top
dma_rx_fsm
dma_rx_phase_ctrl
dma_rx_len_detect
dma_rx_ctrl_proc
dma_rx_data_proc
dma_rx_parity_check
dma_rx_write
dma_crc_pkg
lock_valid_threshold
rand in TB - via config_db
Output agents
Data agent
monitors data_out (32-bit)
Control agent
monitors control_out (16-bit)
Parity agent
parity_valid · parity_not_valid
Data scoreboard
predicted vs observed (32-bit)
Control scoreboard
predicted vs observed (16-bit)
Parity scoreboard
predicted vs observed (1-bit)
Coverage subscriber
length · control · phase
CRC errors · lock threshold
10-transaction sequence
byte stream
Reference model
FSM: NOT_LOCKED
MAYBE_LOCKED - LOCKED
CRC - phase detect - predict
Legend
Sequences
Input agent
DUT
Output agents
Reference model
Scoreboards
Coverage
Data flow
Byte stream to RM
Agent -> scoreboard
RM -> scoreboard
Coverage metadata

- Each scoreboard receives predicted items from the reference model and observed items from the relevant output agent, and compares them.
- The coverage subscriber receives per-packet metadata from the reference model and samples the functional covergroup.

# 3. Agents

The environment uses a single generic DV agent (dv_agent) parameterised by DATA_WIDTH and SEQUENCE_ITEM_WIDTH. The same agent class handles both driving and monitoring by toggling is_master.

## 3.1 Input DV agent (master, DATA_WIDTH = 4)

### Sequencer

Pulls items from the active sequence and forwards them to the driver one at a time.

### Driver

Receives an 8-bit byte and splits it into two 4-bit nibbles. For each nibble the valid signal is randomised each cycle according to valid_probability. Three operating modes will be supported:

- Full throughput — valid every cycle (valid_probability = 100)
- Sparse random — valid 25% of cycles (valid_probability = 25)
- Deterministic — 1-in-4 fixed pattern: valid, idle, idle, idle (valid_probability = 1)

The active mode is selected once per test run and distributed via config_db.

### Monitor

Watches the same interface as the driver. Collects nibbles and reassembles them into bytes, then broadcasts each byte to the reference model via an analysis port. The monitor is always active regardless of is_master.

## 3.2 Output agents (slave, monitor-only)

Three slave agents observe the DUT outputs. Each has only a monitor; no driver or sequencer is instantiated.

| Agent | DATA_WIDTH | Signals monitored | Item produced |
|---|---|---|---|
| Output data agent | 32 | data_out, output_valid | 32-bit data word per valid cycle |
| Control agent | 16 | control_out, control_valid | 16-bit control value per valid cycle |
| Parity agent | 1 | parity_valid, parity_not_valid | 1-bit indication per packet end |

**Parity agent wiring**
The DUT drives two separate 1-bit signals. The testbench combines them:
parity_if.data[0] = parity_not_valid
parity_if.valid = parity_valid | parity_not_valid
So data[0] = 0 means parity OK, data[0] = 1 means parity error.

# 4. Sequences and Items

## 4.1 rx_item — packet stimulus

rx_item represents a complete packet. The item will be randomised by the UVM engine; post_randomize() will compute CRC values and the parity field automatically, and then apply error injection according to the error flags.

| Field | Type | Description |
|---|---|---|
| length | rand [15:0] | Packet length in bytes (range 4–8192, multiples of 4). Weighted towards 4 and 4096. |
| control | rand [15:0] | Control field value. Weighted towards 0. |
| data[ ] | rand byte[ ] | Payload bytes. Size computed from length after randomisation. |
| length_crc | [15:0] | Auto-computed by post_randomize using CRC16 (poly 0x1021). |
| control_crc | [15:0] | Auto-computed by post_randomize using CRC16 (poly 0x8005). |
| parity | [31:0] | Auto-computed: running XOR of all 32-bit data words. |
| length_error | rand bit | If 1 — flip exactly one bit in the transmitted {length, length_crc} word. |
| wrong_length | rand bit | If 1 — flip two or more bits in {length, length_crc} (uncorrectable). |
| control_error | rand bit | If 1 — flip exactly one bit in {control, control_crc}. |
| parity_error | rand bit | If 1 — corrupt the parity field (odd number of bit-flips). |
| half_packet | rand bit | If 1 — transmit only a partial packet, used to create junk and phase changes. |

## 4.2 rx_input_data_sequence

The sequence randomises an rx_item using the active constraints, packs it into a bit-array, and sends 8-bit chunks to the sequencer. Any leftover bits that do not complete a full byte are saved in a static variable and prepended to the next sequence invocation, correctly modelling a continuous bit stream across packet boundaries.

## 4.3 rx_virtual_sequence — test scenario controller

The virtual sequence orchestrates the full test. It runs in three phases:

### Phase 1 — Initial junk

Before attempting lock, the sequence sends one half_packet of random size. This creates a random byte-phase offset in the bit stream so the DUT must always search for lock rather than starting perfectly aligned.

### Phase 2 — Lock acquisition

The sequence selects one of the following lock scenarios (lock_type):

| lock_type | Description | Expected DUT state |
|---|---|---|
| LOCK_REGULAR | Send LVT consecutive valid packets with correct CRC. The fastest path to lock. | LOCKED |
| LOCK_ONE_PACKET_OUT | Send LVT valid packets (achieving lock), then one additional packet whose length is not the minimum value. Tests DUT behaviour immediately after lock. | LOCKED |
| LOCK_PHASE_CHANGE | Send 1 to LVT-1 valid packets, inject a half_packet to shift the phase, then send LVT more valid packets. Tests phase re-detection. | LOCKED |

| lock_type | Description | Expected DUT state |
|---|---|---|
| LOCK_MULTI_BIT_ERR | Send 1 to LVT-1 valid packets, inject a wrong_length packet (2+ bit errors), then send LVT valid packets. Tests recovery from an uncorrectable error. | LOCKED |

#### Phase 3 — Test body

After lock acquisition the sequence selects one of two traffic patterns (test_type):

| test_type | Description |
|---|---|
| REGULAR | Send a randomised number of packets (100–1000) with random lengths and error probabilities driven by the generation parameters. Fast to simulate. |
| REAL | Simulate a real-world traffic pattern: each transaction consists of many minimum-size packets, a batch of optimal-size (4 KB) packets, and one mid-size packet. Repeated for 10–100 transactions. Long to simulate but high coverage value. |

### 4.4 Generation parameters

A configuration object (rx_generation_parameters) holds all probability weights. It is randomised once per test run and distributed to all sequences via config_db. Key parameters:

| Parameter group | Controls |
|---|---|
| Test type weights | Relative probability of REGULAR vs REAL test |
| Lock type weights | Relative probability of each lock_type scenario |
| Valid probability | Distribution across the three driver throughput modes |
| Packet count limits | Min/max packets in REGULAR; min/max transactions in REAL |
| Error probabilities | Per-packet likelihood of length_error, wrong_length, control_error, parity_error |
| Control zero weight | Probability that the control field is zero (→ suppressed on output) |

## 5. Reference Model

The reference model (rx_reference_model) is a transaction-level predictor. It receives the same byte stream as the DUT (via the input monitor), runs an identical FSM, and emits expected transactions to the three scoreboards and coverage metadata to the coverage subscriber.

### 5.1 FSM — three lock states

The model maintains three lock states that mirror the DUT behaviour:

| State | Behaviour | Transitions |
|---|---|---|
| NOT_LOCKED | Maintain a sliding window of bytes. For every new byte, check all 8 possible bit-phase offsets (shifts 0–7 bits) against the expected {length, length_crc} CRC. The window is FRAME_WIDTH bits wide to cover all offsets simultaneously. | CRC valid at any phase → MAYBE_LOCKED |

| State | Behaviour | Transitions |
|---|---|---|
| MAYBE_LOCKED | Parse packets at the detected phase. For each packet, check the length CRC. Zero or one-bit errors are corrected and tolerated. Two or more bit errors reset the model to NOT_LOCKED. Count consecutive valid packets; reach LVT → LOCKED. | LVT consecutive valid packets → LOCKED<br>2+ bit errors in length → NOT_LOCKED |
| LOCKED | Parse and forward packet contents. Length CRC errors of 0 or 1 bit are corrected. Two or more bit errors in length are treated as a fatal condition. Control CRC: 0 or 1 bit errors corrected before forwarding; if control value is zero, no item is emitted. Data words are forwarded every 4 bytes. Parity is verified at end of packet. | 2+ errors in length → fatal |

### 5.2 Packet processing in LOCKED state

Once locked, the model processes each incoming byte through four sub-phases:

| Sub-phase | Bytes consumed | Model action | Output emitted |
|---|---|---|---|
| PACKET_LENGTH | 4 | Accumulate {length[15:0], length_crc[15:0]}. Apply CRC16 (poly 0x1021). Correct single-bit errors. | expected_msg_length — used for subsequent phases |
| PACKET_CONTROL | 4 | Accumulate {control[15:0], control_crc[15:0]}. Apply CRC16 (poly 0x8005). Correct single-bit errors. If corrected value is non-zero, emit a control item. | control item → control scoreboard (if != 0 and <= 1 error) |
| PACKET_DATA | length − 4 | Every 4 bytes: emit a 32-bit data word and XOR it into the running parity accumulator. | data item (32-bit word) → data scoreboard, every 4 bytes |
| PACKET_PARITY | 4 | XOR the running accumulator with the 4 incoming parity bytes. Compare to zero. | parity item (0 = OK, 1 = error) → parity scoreboard |

### 5.3 Phase detection

In NOT_LOCKED the model maintains a shift register (window_frame) wide enough to hold one full {length + length_crc} word at any of the 8 possible bit offsets. Every new byte is shifted in. Once the window is full, all 8 slices are tested against the CRC. The first slice that passes becomes the locked phase (0–7). This phase is then used in MAYBE_LOCKED and LOCKED to extract aligned bytes from the raw nibble stream.

### 5.4 Single-bit error correction

In MAYBE_LOCKED and LOCKED the model applies a brute-force single-bit correction: each of the 32 bits of the {data, CRC} word is flipped in turn and the CRC is re-evaluated. If exactly one flip produces a valid CRC, the corrected word is used and num_errors is set to 1. If no single flip succeeds, num_errors is set to 2 (uncorrectable), and the model acts according to the state rules above.

## 6. Scoreboards

Three instances of a generic scoreboard (rx_scoreboard #(DATA_WIDTH)) will compare reference model predictions against DUT observations. All three share the same comparison mechanism.

### 6.1 Comparison mechanism

Each scoreboard has two analysis ports: env_port (from the reference model) and dut_port (from the relevant output agent monitor). Items arriving at either port are pushed onto separate queues. Whenever both queues are non-empty, the front items are popped and compared field-by-field. A mismatch triggers a UVM fatal error. At end-of-test, if either queue is non-empty the scoreboard holds the run_phase until all items are drained and compared.

### 6.2 Three scoreboard instances

| Instance | DATA_WIDTH | What is compared |
|---|---|---|
| Data scoreboard | 32 | Every 32-bit data word emitted by the DUT while locked |
| Control scoreboard | 16 | Every non-zero 16-bit control value emitted by the DUT |
| Parity scoreboard | 1 | Parity indication (0 = OK, 1 = error) at the end of each packet |

## 7. Functional Coverage

The coverage subscriber (rx_coverage) receives a coverage item from the reference model at the end of every packet and at state-transition events. The following coverpoints and crosses must be closed for the test suite to pass.

### 7.1 Packet-level coverpoints

| Coverpoint | What is measured | Required bins |
|---|---|---|
| Packet length | Distribution of packet sizes across the full supported range | Minimum (4 B), several 1 KB bands, optimal (4 KB), several upper bands, maximum (8192 B) |
| Control value | Whether the control field was zero or non-zero | control = 0 / control != 0 |
| Length CRC errors | Number of bit errors detected in the {length, length_crc} word | 0 errors / 1 error / 2+ errors |
| Control CRC errors | Number of bit errors detected in the {control, control_crc} word | 0 errors / 1 error |
| Parity result | Whether the packet parity was valid or erroneous | OK (0) / error (1) |
| Locked phase | The bit-offset at which lock was achieved (0–7, where phase N equals phase N+4) | Four phase pairs: {0,4}, {1,5}, {2,6}, {3,7} |
| Lock threshold | The randomised value of lock_valid_threshold for this run | One bin per value in the range MIN..MAX |

### 7.2 Scenario-level coverpoints

| Coverpoint | What is measured | Required bins |
|---|---|---|
| Lock achieved | A MAYBE_LOCKED → LOCKED transition was observed | at least one transition seen |
| MAYBE drop — early | Model dropped from MAYBE_LOCKED to NOT_LOCKED with fewer than LVT-1 valid packets accumulated | at least one early drop seen |
| MAYBE drop — at LVT-1 | Model dropped from MAYBE_LOCKED to NOT_LOCKED having accumulated exactly LVT-1 packets | at least one such drop seen |
| Ten-transaction sequence | At least 10 sequences of OPTIMAL → mid-size packets were observed during a single run | flag = 1 |

| Coverpoint | What is measured | Required bins |
|---|---|---|
| Valid probability mode | All three driver throughput modes exercised | 100% / 25% / 1-in-4 ratio |

### 7.3 Required cross coverage

| Cross | Axes | Purpose |
|---|---|---|
| MAYBE drop × lock threshold | maybe_dropped_consecutive_count × lock_threshold | Ensure that drops at various accumulation depths are seen across the full threshold range. Impossible combination (early drop when LVT=2) is excluded. |

## 8. Key Parameters

| Parameter | Value | Description |
|---|---|---|
| Input nibble width | 4 bit | Width of input_data port |
| Output data width | 32 bit | Width of data_out port |
| Control field width | 16 bit | Width of control_out port |
| Length field width | 16 bit | Bits used to encode packet length |
| Min packet length | 4 bytes | Shortest valid packet (control only, no data) |
| Max packet length | 8192 bytes | Longest valid packet |
| Optimal packet length | 4096 bytes | Target size for the ten-transaction coverage scenario |
| Phase search width | 8 | Number of bit-offset phases checked during lock search |
| Lock threshold range | 2 – 12 | Allowed values for lock_valid_threshold |
| Length CRC polynomial | 0x1021 | CRC-CCITT polynomial used for length field |
| Length CRC initial value | 0xFFFF | CRC-CCITT initial value |
| Control CRC polynomial | 0x8005 | CRC-16/IBM polynomial used for control field |
| Control CRC initial value | 0x0000 | CRC-16/IBM initial value |
| Clock period | 10 ns | 100 MHz system clock |